

called views, and files that typically serve to modify the manner in which the views and transitions appear. Views have many different subclasses that you can explore and offer a wide variety of uses. By going through a few sample cases you can find that you can modify when a view appears, for how long it stays on the UI and also how it responds to user inputs. The subclasses are quite specialised in function and behaviour.

View controllers on iOS come in a variety of different flavors. These controllers offer a multitude of options to the developer to not only customise the way the app looks but also the way it functions. The available categories are:

### **Standard view controllers**

These present a view, and nothing more.

### **Navigation controllers**

These present a stack of view controllers, onto which the application can push additional view controllers.

### **Table view controllers**

These present a list of cells that can be individually configured, stylized, ordered, and grouped. They present a single column of cells with individual customisability.

### **Tab bar controllers**

These present a set of view controllers, selectable through a tab bar at the bottom of the screen.

### **Split view controllers**

These present a side-by-side parent and child view structure, allowing you to see an overview in the parent view and detailed information in the child view.

### **Page controllers**

These present view controllers in a “page-turning” interface, similar to the iBooks application on the iPad and iPhone.

There are many others as well, but the ones presented above provide enough of a starting point for the beginning app developer.

## **Creating items using library files**

Let's begin by adding a simple text field to the scene and mess around with